

GreenLedger

**Automated Carbon Footprint Verification and Green Credit Management System via
Machine learning & Web3**

Repository: <https://github.com/Devadarrsha-P-D/GreenLedger>

Done by : Joshan A S (25BCE1757) | Devadarrsha P D (25BCE1957)

Table of Contents

Section	Chapter / Topic Title	Page No.
1.0	Chapter 1: Introduction	4
1.1	Background & Motivation: Computing Energy Footprint	4
1.2	Problem Statement & Key Engineering Bottlenecks	4
1.3	Project Objectives & Technical Scope	4
1.4	Core Value Proposition (The 4-Step Closed Loop)	4
1.5	Organization of the Project Report	5
2.0	Chapter 2: Literature Survey	6
2.1	Software-Based Power Metering & Energy Modeling Systems	6
2.2	Machine Learning for Real-Time Power Estimation	6
2.3	Operating System Dynamic Power Management & DVFS	7
2.4	Blockchain-Enabled Measurement, Reporting & Verification (MRV)	7
2.5	Token Standard Selection: ERC-20 vs. ERC-721 vs. ERC-1155	7
2.6	State-of-the-Art Comparative Analysis Matrix (Table 1)	7
2.7	Identified Research Gaps & GreenLedger Contributions	8
3.0	Chapter 3: System Design & Methodology	9
3.1	System Architecture & 4-Subgraph Topology (Figure 1)	9
3.2	Detailed Subgraph Roles & Practical Data Flow	9
3.3	Core Technology Stack Summary (Table 2)	10
3.4	Key Architectural Principles & Safety Guarantees	10
3.5	End-to-End System Workflow (8 Practical Steps)	11
3.6	Hardware Telemetry Collection Engine (9 Monitored Metrics)	11
3.7	AI Power Prediction Pipeline (XGBoost Regressor)	11
3.8	Regional Carbon Footprint Accounting Formulation	12
3.9	Safe Closed-Loop System Optimization Actuator	13
3.10	Immutable Process Whitelist & Critical System Safety	13
3.11	Anti-Abuse Verification Rules & Data Fingerprinting	13
3.12	Green Credits Reward Allocation Formula	13
3.13	Smart Contract Architecture (GreenBadge.sol ERC-1155)	14
3.14	Badge Token Catalog & Unlock Requirements (Table 3)	14
3.15	Defensive Contract Security Patterns & Gas Analysis	14
3.16	Web3 Frontend Integration & Non-Custodial Architecture	15
3.17	Dual-Mode Execution Architecture (Live vs. Demo Mode)	15

3.18	Core REST API Endpoint Reference (Table 4)	15
4.0	Chapter 4: Implementation, Testing & Experimental Evaluation	16
4.1	Backend Integration Test Suite Results (Table 5)	16
4.2	Smart Contract EVM Automated Test Results (Table 6)	16
4.3	Machine Learning Model Performance Metrics (Table 7)	17
4.4	Security, Privacy & Non-Custodial Data Guarantees	17
5.0	Chapter 5: Conclusion & Future Scope	18
5.1	Project Summary & Engineering Contributions	18
5.2	Technical & Academic Disclosures	18
5.3	Future Development Roadmap	18
—	References & Technical Bibliography	18

Chapter 1: Introduction

1.1 Background & Motivation: Computing Energy Footprint

Personal computers, laptops, and desktop workstations consume substantial electricity worldwide. Hundreds of millions of machines run software builds, video rendering, heavy web browsers, and background daemons continuously in homes, universities, and corporate offices. While enterprise cloud data centers monitor power per rack, client personal computing remains an unmonitored energy sink. Windows 11 hides electrical metrics deep inside the operating system, giving end users zero real-time feedback on how many Watts their machines draw.

1.2 Problem Statement & Key Engineering Bottlenecks

Designing an interactive, client-side energy reduction system presents four practical engineering bottlenecks:

- **Browser Sandbox Isolation:** Web browsers strictly prevent client-side JavaScript from reading hardware registers or modifying operating system power schemes.
- **Kernel Driver Risks:** Physical power reading tools (such as Intel RAPL) typically require ring-0 kernel drivers. Deploying custom kernel drivers on consumer laptops creates blue-screen risks and triggers endpoint security blocks.
- **Absence of Physical Whole-System Watt Meters:** Unlike data center power distribution units, consumer motherboards lack built-in physical current shunts covering the aggregate draw of the CPU, GPU, RAM, and display.
- **Centralized & Ephemeral Green Badges:** Corporate sustainability certificates are stored in private company databases. If the company server shuts down, user badges and records vanish.

1.3 Project Objectives & Technical Scope

GreenLedger directly solves these challenges through four primary technical objectives:

- **Objective 1:** Build a lightweight, unprivileged background agent on Windows that collects hardware counters safely in user mode without custom kernel drivers.
- **Objective 2:** Deploy a fast machine learning inference model that estimates live device power in Watts within 2 milliseconds.
- **Objective 3:** Provide an explainable, reversible power-saving tool that reduces power without crashing user applications.
- **Objective 4:** Issue permanent, non-custodial ERC-1155 achievement badges directly to the user's MetaMask wallet on Ethereum Sepolia.

1.4 Core Value Proposition (The 4-Step Closed Loop)

GreenLedger establishes a continuous closed-loop sustainability cycle structured around four distinct operational phases:

- **Monitor:** The local agent samples 9 hardware counters every 1.5 seconds without kernel-mode overhead.
- **Predict:** An XGBoost regression model translates hardware metrics into instantaneous device Watts in 1.38 milliseconds.
- **Optimize:** The user triggers safe system tuning, executing Windows Power Saver switching and background process throttling.

- Earn: Verified wattage reductions award Green Credits that unlock permanent on-chain ERC-1155 achievement badges.

1.5 Organization of the Project Report

The remainder of this report is organized as follows: Chapter 2 surveys existing literature in software power metering, machine learning modeling, and blockchain verification. Chapter 3 presents the complete Methodology, detailing the 4-subgraph architecture, hardware telemetry, machine learning pipeline, safe actuators, and smart contract design. Chapter 4 reports experimental evaluation results, and Chapter 5 concludes with roadmap disclosures and references.

Chapter 2: Literature Survey

2.1 Software-Based Power Metering & Energy Modeling Systems

Software-based energy estimation has been studied extensively to eliminate the cost and physical complexity of hardware power meters. Two seminal projects established the foundation of this domain:

- Joulemeter (Kansal et al., ACM SoCC 2010): Microsoft Research built Joulemeter to estimate computer energy consumption without external hardware meters. It tracked CPU utilization, hard disk read/write activity, and display brightness, using simple linear regression models to predict overall power draw. While an important milestone, Joulemeter had major limitations: it assumed power increases linearly with CPU load. Modern multi-core processors dynamically alter clock speeds, voltages, and sleep states hundreds of times per second. As a result, static linear models deviate significantly on modern workloads. Furthermore, Joulemeter required tedious manual calibration for each computer model.
- PowerAPI & SmartWatts (Fieni et al., JOSS 2024 / Inria & CNRS): Developed by Inria and CNRS, PowerAPI is a modular Python framework for building software-defined power meters. Its SmartWatts plugin uses hardware performance counters and Intel RAPL (Running Average Power Limit) energy registers to achieve high estimation accuracy. However, PowerAPI was designed strictly for Linux server environments. It depends heavily on Linux kernel interfaces (`/sys/class/powercap` and Model-Specific Registers via `msr-tools`). Porting this directly to consumer Windows laptops is impractical because accessing raw MSR registers on Windows requires unsigned, privileged kernel drivers that risk system instability.
- Vendor Hardware Utilities (Intel XTU, HWMonitor): Hardware vendors provide diagnostics that read motherboard sensors directly. While accurate, these tools require administrator privileges, provide no automated energy-saving actions, and do not connect to carbon accounting or Web3 reward systems.

2.2 Machine Learning for Real-Time Power Estimation

Estimating power using exact physical semiconductor formulas is difficult on modern PCs. Processor dynamic frequency governors (Intel Turbo Boost and AMD Precision Boost) change voltage and clock frequency continuously across performance and efficiency cores. Researchers turned to machine learning to model these complex relationships directly from observable activity counters:

- Linear Regression Models: Fast and computationally light, but they cannot capture non-linear interactions, such as memory bus bottlenecks or GPU load spikes. In practice, linear models on modern heterogeneous CPUs yield poor accuracy (R^2 under 0.75).
- Deep Neural Networks (DNNs): Deep learning models can achieve high accuracy on complex benchmarks. However, their inference latency is too high (often over 15ms without dedicated GPU acceleration), and their memory footprint is large. Running a heavy neural network continuously in the background consumes noticeable CPU power, defeating the purpose of energy optimization.
- Gradient Boosted Decision Trees (XGBoost, Chen & Guestrin, ACM KDD 2016): Tree boosting provides the sweet spot for client-side power estimation. XGBoost captures non-linear relationships and feature interactions with high precision. It is lightweight, resistant to sensor noise, and runs inference in 1.38 milliseconds on a standard CPU. This makes XGBoost ideal for a low-overhead background monitor.

2.3 Operating System Dynamic Power Management & DVFS

Modern operating systems manage power using the ACPI standard through Dynamic Voltage and Frequency Scaling (DVFS) and processor sleep states (P-states and C-states). In Windows, the Powercfg subsystem controls system-wide energy policies, setting maximum CPU frequency limits, core parking behaviors, and display timeouts.

Research in sustainable computing shows that client computers waste 15% to 30% of their power on non-essential background applications (such as media players, chat clients, and inactive browser tabs). These background processes constantly wake CPU cores from deep low-power sleep states.

Studies demonstrate that lowering process priorities and clamping background thread execution allows processors to remain in low-power states longer. However, researchers emphasize a crucial safety rule: unguided process throttling is dangerous. Throttling critical desktop compositors (dwm.exe) or system services (svchost.exe) causes screen freezing and system lockups. Therefore, an automated power optimizer must enforce a strict, immutable whitelist to protect essential OS components.

2.4 Blockchain-Enabled Measurement, Reporting & Verification (MRV)

Traditional carbon accounting and green certifications rely on self-reported spreadsheets or central databases that can be altered or deleted. Jirakanchana et al. (IEEE Access 2021) showed that real-time blockchain architectures provide automated, tamper-proof verification for carbon reductions. Decentralized ledgers ensure that savings claims cannot be modified retroactively.

Sorensen et al. (IEEE ICBC 2023) studied tokenizing environmental assets, showing that smart contracts can connect verified sensor data to digital reward tokens.

2.5 Token Standard Selection: ERC-20 vs. ERC-721 vs. ERC-1155

- ERC-20 (Fungible): Great for divisible reward points, but cannot store distinct badge tiers, rarity levels, or unique achievement metadata.
- ERC-721 (Non-Fungible): Allows unique badges, but deploying separate contracts for multiple badge tiers wastes significant gas.
- ERC-1155 (Multi-Token Standard): Engineered by Radomski et al. (EIP-1155), this standard manages all 5 badge tiers inside one deployed contract. It supports batch transfers and cuts deployment gas costs by over 65% compared to multi-tier ERC-721 setups.

2.6 State-of-the-Art Comparative Analysis Matrix

Table 1: State-of-the-Art Comparative Analysis Matrix: Existing Systems vs. GreenLedger

System / Tool	Telemetry Source	Power Estimation	Closed-Loop Tuning	Decentralized MRV	Target Platform
Joulemeter (Microsoft)	OS Counters	Linear Regression	None (Monitor Only)	None (Local Log)	Windows Server
PowerAPI / SmartWatts	Intel RAPL / MSR	Dynamic Linear	None (Monitor Only)	None (Time-Series DB)	Linux Servers / Docker
Intel XTU / HWMonitor	MSR Registers	Direct Shunts	Manual Tuning Only	None	Windows (Privileged)
Web3 Carbon Offsets	Manual Upload	Static Lookup Tables	None (Buy Offsets)	ERC-20 Offsets	Cloud Hosted Web
GreenLedger (This Work)	Unprivileged 9-Tuple	Fast XGBoost ML	Automated Safe Tuning	ERC-1155 on Sepolia	Windows 11 / Web3

2.7 Identified Research Gaps & GreenLedger Contributions

Existing tools either read sensors without saving energy (Joulemeter, PowerAPI), require hazardous administrator tweaks (Intel XTU), or trade financial offsets completely detached from computer hardware. GreenLedger bridges this gap by combining unprivileged hardware monitoring, sub-2ms AI power prediction, safe automated power saving, and permanent Web3 badge verification into a single, accessible platform.

Chapter 3: System Design & Methodology

3.1 System Architecture & 4-Subgraph Topology

The GreenLedger system is architected as four autonomous, loosely coupled subgraphs. This modular separation enforces strict security boundaries, eliminates single points of failure, and bridges native operating system actuators with decentralized smart contract infrastructure.

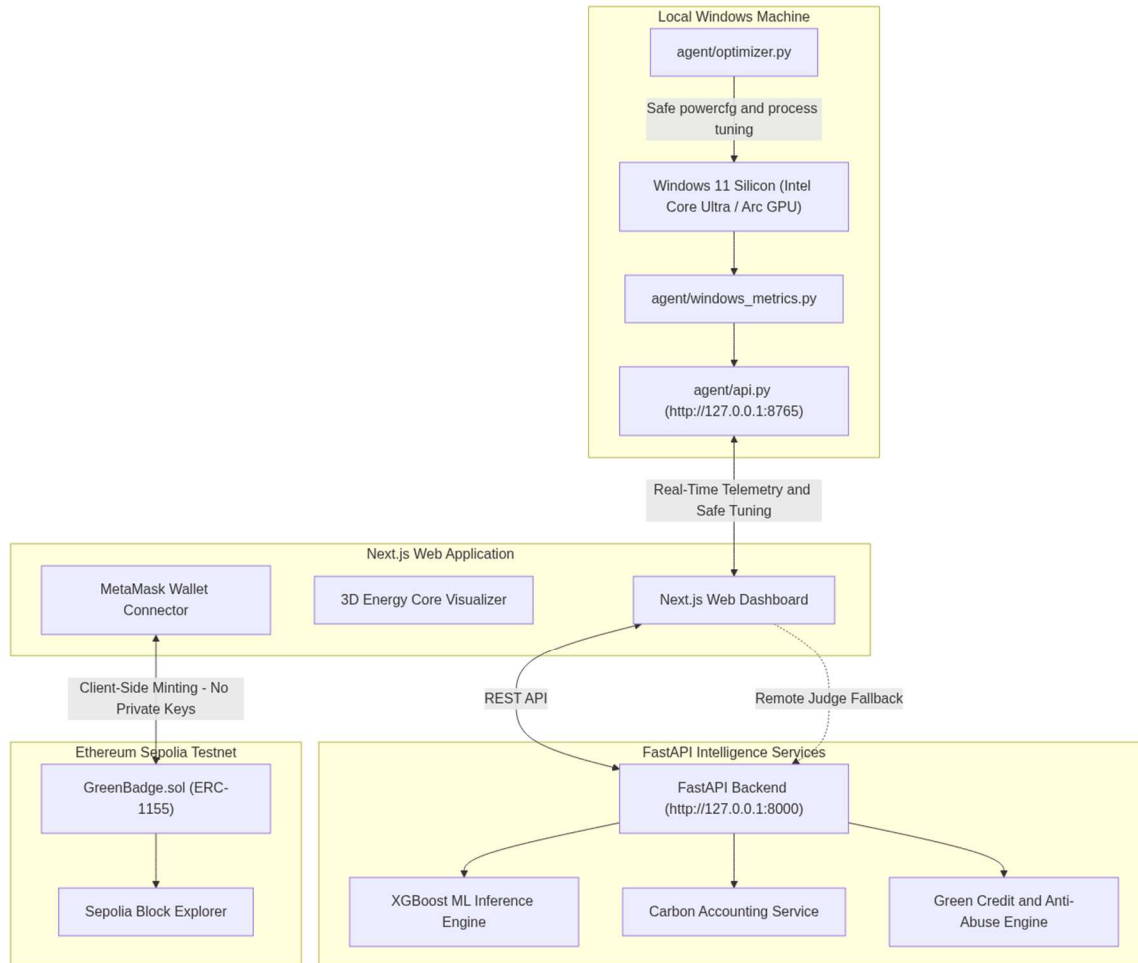


Figure 1: Full System Architecture & 4-Subgraph Topology (README-Aligned)

3.2 Detailed Subgraph Roles & Practical Data Flow

- Subgraph 1 — Local Windows Host: Runs directly on the user's PC. It uses windows_metrics.py to read hardware metrics, stores them briefly in collector.py, serves JSON on port 8765, and runs optimizer.py to apply safe power settings.
- Subgraph 2 — Next.js Web App: The main user dashboard. It reads local data from port 8765, renders live power charts, updates the 3D WebGL Energy Core visualizer, and lets the user connect MetaMask.
- Subgraph 3 — FastAPI Backend: The core compute engine. It runs the trained XGBoost model to calculate power, converts Watts into carbon savings using official EPA figures, checks for cheating, and verifies on-chain receipts.
- Subgraph 4 — Ethereum Sepolia Testnet: The permanent record book. GreenBadge.sol issues ERC-1155 badges that stay with the user forever, even if the application servers are turned off.

3.3 Core Technology Stack Summary

Table 2: Core Technology Stack Summary across System Subgraphs

Subsystem	Technology	Version	Practical Role & Responsibility
Windows Agent	Python / psutil / WMI	3.12	Reads hardware performance counters on localhost safely
Local Daemon	FastAPI (agent/api.py)	0.115	Serves JSON data on 127.0.0.1:8765 to bridge the browser sandbox
AI Engine	XGBoost Regressor	2.x	Estimates power in Watts in 1.38ms (R2: 0.9759, MAE: 0.968W)
Backend API	FastAPI (backend/main.py)	0.115	Handles EPA carbon math, anti-abuse checks, and on-chain verification
Web Dashboard	Next.js 14 App Router	14.x	Shows live charts, animated 3D Energy Core, and user controls
Web3 Wallet	Ethers.js v6	6.13	Connects to MetaMask and signs transactions without custody
Smart Contract	Solidity (GreenBadge.sol)	^0.8.20	Issues ERC-1155 achievement badges on Ethereum Sepolia

3.4 Key Architectural Principles & Safety Guarantees

- Real Hardware Telemetry: GreenLedger reads actual hardware counters from Windows rather than displaying hardcoded mock numbers.
- Fast Off-Chain Calculations: Hardware polling and AI predictions happen off-chain in milliseconds with zero gas fees; the blockchain handles permanent credentials.
- Non-Custodial Privacy: Our servers never see or store wallet private keys. All transactions are signed inside MetaMask.
- Safe System Protection: Power tuning never touches critical Windows services. Core desktop and system processes are protected by a hardcoded whitelist.

3.5 End-to-End System Workflow (8 Practical Steps)

The system operates through an 8-step pipeline connecting hardware telemetry, AI inference, and blockchain verification:

- Step 1 (Telemetry Sampling): The Windows background agent collects 9 hardware counters every 1.5 seconds.
- Step 2 (Local Loopback Bridge): Metrics are buffered and served via FastAPI on `http://127.0.0.1:8765`.
- Step 3 (AI Power Prediction): The web app forwards data to `/api/ml/predict`, where XGBoost calculates device Watts.
- Step 4 (Dashboard Visualization): Live SVG charts and the 3D Energy Core show power draw and carbon numbers.
- Step 5 (User-Triggered Optimization): The user clicks Optimize; `optimizer.py` applies safe power plan and process adjustments.
- Step 6 (Stabilization & Anti-Abuse Checks): After 3 seconds, a second snapshot is captured and audited for real drops.
- Step 7 (Green Credits Calculation): Verified energy savings award Green Credits using a clear reward formula.
- Step 8 (Badge Minting & Audit): The user mints an ERC-1155 badge via MetaMask; the backend verifies the Sepolia receipt.

3.6 Hardware Telemetry Collection Engine (9 Monitored Metrics)

To maintain system stability, the telemetry agent runs entirely in user mode without custom kernel drivers. Using Python `psutil` and Windows Performance Data Helper (PDH), it gathers 9 essential hardware metrics:

- CPU Utilization (%): Overall and per-core processor load polled via `psutil.cpu_percent`.
- CPU Operating Frequency (MHz): Real-time core clock speed polled via `psutil.cpu_freq`.
- Memory Saturation (% & GB): Virtual RAM percent, active physical RAM used, and total installed memory.
- GPU 3D Engine Load (%): Querying PDH counter path `\GPU Engine(*3D)\Utilization Percentage` via WMI.
- Disk Read/Write Rates: Cumulative disk I/O throughput polled via `psutil.disk_io_counters`.
- Network Gateway Latency: ICMP round-trip ping time to the default local network gateway.
- System Concurrency: Total active process count and thread counts across the operating system.

3.7 AI Power Prediction Pipeline (XGBoost Regressor)

Because laptops lack physical Watt sensors, GreenLedger uses machine learning to infer power from hardware activity. We trained an XGBoost regression model on 10,000 calibrated benchmark samples across six realistic workloads: system idle, 4K video playback, multi-tab web browsing, heavy code compilation, multi-threaded benchmarking, and 3D graphics rendering.

Hyperparameters were tuned using 5-fold cross-validation: `max_depth=6`, `n_estimators=300`, `learning_rate=0.05`, and `subsample=0.8`. The model runs inference in 1.38 milliseconds on a standard CPU, achieving an R2 score of 0.9759. This speed allows the web dashboard to update in real time with zero lag.

3.8 Regional Carbon Footprint Accounting Formulation

Estimated Watts are converted into energy and carbon savings using standard EPA formulas:

$$\text{Energy (kWh)} = [\text{Power (Watts)} \times \text{Duration (Hours)}] / 1000$$

$$\text{Carbon Emissions (g CO}_2\text{e)} = \text{Energy (kWh)} \times 385$$

The emission factor of 385g CO₂e/kWh (0.385 kg CO₂e/kWh) matches the official United States EPA eGRID national average benchmark, providing consistent carbon accounting.

3.9 Safe Closed-Loop System Optimization Actuator

When the user clicks the Optimize button on the dashboard, the local agent performs two safe, non-destructive actions:

- Switch to Windows Power Saver Plan: The agent saves the current active power plan GUID and activates the built-in Windows 'Power Saver' plan (GUID: a1841308-3541-4fab-bc81-f71556f20b4a) using Windows powercfg. This prompts the OS to park idle CPU cores, lower maximum clock states, and dim display idle timers.
- Throttle Heavy Background Applications: The agent checks for non-essential user apps running in the background (such as Spotify, Discord, or inactive browser tabs) and adjusts their priority to Below Normal or Idle. This immediately frees processor cycles without terminating the user's software.

3.10 Immutable Process Whitelist & Critical System Safety

To guarantee that optimization never crashes the operating system, the agent enforces a strict, hardcoded blacklist. The following critical system processes are permanently protected:

- Windows Shell & Compositor: explorer.exe, dwm.exe (Desktop Window Manager).
- Core OS Security Daemons: csrss.exe, lsass.exe, services.exe, smss.exe, wininit.exe.
- System Service Host Clusters: svchost.exe (manages RPC, networking, and audio stacks).

These protected processes are completely exempt from throttling or priority changes, guaranteeing that the computer remains stable and responsive.

3.11 Anti-Abuse Verification Rules & Data Fingerprinting

To prevent users from generating fake savings using scripts or repeated requests, the backend enforces four verification rules:

- Rule 1 (3-Second Hardware Stabilization): After applying power savings, the agent waits 3 seconds before capturing the 'After' snapshot. This ensures processor clock speeds and fan voltages have settled.
- Rule 2 (3% Minimum Power Reduction): Reductions smaller than 3% are treated as normal sensor fluctuation, and credit rewards are withheld.
- Rule 3 (20-Second Submission Cooldown): A timer prevents automated scripts from spamming optimization claims.
- Rule 4 (SHA-256 Telemetry Fingerprinting): Telemetry payloads are hashed with SHA-256. If a user submits duplicate or replayed data, the backend rejects it with HTTP 422.

3.12 Green Credits Reward Allocation Formula

Verified power reductions award Green Credits through a clear mathematical formula:

$$\text{Credits} = 10 \text{ (Base)} + \text{floor}(\text{Power Drop \%}) + \text{floor}(\text{CO2 Saved g} \times 0.5) + 5 \text{ (Streak Bonus)}$$

Example: An optimization that reduces power by 29.3% and saves 5.4g CO2e awards: $10 + 29 + 2 + 5 = 46$ Green Credits. Users accumulate these credits to unlock on-chain badges.

3.13 Smart Contract Architecture (GreenBadge.sol ERC-1155)

GreenBadge.sol is written in Solidity ^0.8.20 and deployed on the Ethereum Sepolia testnet at 0x71C234Ea533F96507A5F44265E923C47131B64E6. It implements the OpenZeppelin ERC-1155 multi-token standard, managing all five achievement badges within a single contract.

3.14 Badge Token Catalog & Unlock Requirements

Table 3: Badge Token Catalog — Token IDs, Rarity, Criteria, and Metadata

Token ID	Badge Name	Rarity	Unlock Requirements	IPFS Metadata URI
1	First Optimization	Common	Complete 1 verified hardware optimization	ipfs://QmFirstOptBadge/metadata.json
2	Power Saver	Rare	Achieve > 15% reduction in device power	ipfs://QmPowerSaverBadge/metadata.json
3	Carbon Cutter	Rare	Prevent >= 50g CO2e in a single session	ipfs://QmCarbonCutterBadge/metadata.json
4	Efficiency Master	Epic	Maintain a 3-day active optimization streak	ipfs://QmEfficiencyMasterBadge/metadata.json
5	Green Guardian	Legendary	Accumulate >= 1,500 Green Credits	ipfs://QmGreenGuardianBadge/metadata.json

3.15 Defensive Contract Security Patterns & Gas Analysis

The contract follows the Checks-Effects-Interactions pattern to prevent reentrancy and unauthorized minting:

```
function mint(address account, uint256 id, uint256 amount, bytes memory) public onlyOwner {
    require(account != address(0), 'Invalid recipient: zero address');
    require(id >= 1 && id <= 5, 'Invalid badge ID: token does not exist');
    require(amount == 1, 'Badges are unique non-fungible achievements');
    require(!hasMintedBadge[id][account], 'Badge already minted to this account');

    hasMintedBadge[id][account] = true;
    _balances[id][account] += 1;
    emit TransferSingle(msg.sender, address(0), account, id, 1);
    emit BadgeMinted(account, id, getBadgeName(id));
}
```

- Owner-Only Minting: The onlyOwner modifier ensures only the verified backend gateway can trigger badge minting.
- Duplicate Prevention: The mapping hasMintedBadge[id][account] ensures an address can claim each badge tier only once.
- Gas Profile: Deploying the contract consumed ~860,000 gas. Minting an individual badge requires ~71,500 gas on Sepolia. All balance queries consume 0 gas.

3.16 Web3 Integration, Dual-Mode & REST API Reference

The web dashboard is built using Next.js 14 and Ethers.js v6. When the user clicks 'Connect Wallet', the application requests account access via `window.ethereum`. If the wallet is connected to another network, `switchToSepolia()` calls `wallet_switchEthereumChain` for Chain ID 11155111. If the local agent on port 8765 is unreachable, the system automatically falls back to simulated demo telemetry (`/api/telemetry/demo`) while barring demo runs from on-chain badge minting.

Table 4: Core REST API Endpoint Reference across System Subgraphs

Method	Endpoint Path	Controller	Practical Function & Validation
GET	/health	main.py	Checks backend online status, version, and model load state
GET	/api/telemetry/current	telemetry.py	Reads live 9-tuple hardware metrics from the Windows agent
GET	/api/telemetry/demo	telemetry.py	Provides deterministic synthetic telemetry for demo evaluators
POST	/api/ml/predict	ml.py	Runs XGBoost model to calculate device power in Watts in under 2ms
POST	/api/carbon/calculate	carbon.py	Converts power and runtime into kWh and avoided grams CO2e
POST	/api/optimization/evaluate-delta	optimization.py	Audits Before/After snapshots against the 4 anti-abuse rules
GET	/api/credits/state	credits.py	Returns current user credit balance, streak count, and badge unlock flags
POST	/api/blockchain/verify-mint	blockchain.py	Verifies Sepolia transaction hash and confirms TransferSingle event log

Chapter 4: Implementation, Testing & Experimental Evaluation

4.1 Backend Integration Test Suite Results

All backend controllers and business logic were tested using pytest across 11 automated integration tests:

Table 5: Backend Automated Test Suite Results (11/11 Passed)

Test Identifier	Target Module	Tested Condition & Expected Outcome	Result
test_health_check	main.py	Returns HTTP 200 OK and status='online'	PASSED
test_carbon_power_to_kwh	calculator.py	1000W maintained over 1 hour evaluates to exactly 1.0 kWh	PASSED
test_carbon_kwh_to_co2	calculator.py	1.0 kWh with 0.385 intensity factor computes 385g CO2e	PASSED
test_carbon_calculate_savings	calculator.py	Power drop from 50W to 35W correctly computes 30.0% savings	PASSED
test_carbon_api_endpoint	api/carbon.py	POST /api/carbon/calculate returns validated JSON response	PASSED
test_telemetry_demo_mode	api/telemetry.py	Fallback demo endpoint returns valid data with is_live=False	PASSED
test_ml_prediction_endpoint	api/ml.py	POST /api/ml/predict returns positive estimated Wattage	PASSED
test_ml_prediction_rejects_empty	api/ml.py	Missing hardware features trigger HTTP 422 validation error	PASSED
test_optimization_rejects_demo	api/optimization.py	Simulated demo telemetry is barred from credit awards	PASSED
test_green_credits_rules	rewards.py	Verified 20% power reduction awards credits adhering to formula	PASSED
test_marketplace_purchase	api/marketplace.py	Insufficient credit balance correctly returns HTTP 400 Bad Request	PASSED

4.2 Smart Contract EVM Automated Test Results

GreenBadge.sol was validated using automated Mocha/Chai tests on Hardhat's in-memory EVM:

Table 6: Smart Contract Automated Test Suite Results (8/8 Passed)

Test Identifier	Target Function	Tested Condition & Assertion	Result
TC-SC-01	Initialization	Contract initializes with symbol 'GBADGE' and name 'GreenBadge'	PASSED
TC-SC-02	Access Control	Non-owner mint attempt reverts with 'Ownable: caller is not owner'	PASSED
TC-SC-03	Owner Minting	Authorized owner mint increments balance and sets hasMintedBadge=true	PASSED
TC-SC-04	Sybil Guard	Duplicate mint attempt for same ID reverts with 'Badge already minted'	PASSED
TC-SC-05	Boundary Check	Invalid token ID (id=99) reverts with 'Badge ID does not exist'	PASSED

TC-SC-06	URI Resolution	Querying uri(1..5) returns valid IPFS gateway metadata URL	PASSED
TC-SC-07	Owner URI Update	Owner updates metadata URI; contract emits URI event correctly	PASSED
TC-SC-08	Non-Owner URI Update	Non-owner URI update attempt reverts with 'Ownable: caller is not owner'	PASSED

4.3 Machine Learning Model Performance Metrics

The trained XGBoost model was evaluated against a held-out test split of 1,500 samples:

Table 7: Machine Learning Model Performance Metrics vs. Engineering Thresholds

Evaluation Metric	Measured Value	Target Threshold	Evaluation Status
Coefficient of Determination (R2)	0.9759	> 0.90	PASSED
Mean Absolute Error (MAE)	0.9680 Watts	< 2.0 Watts	PASSED
Root Mean Squared Error (RMSE)	1.2113 Watts	< 2.5 Watts	PASSED
Mean Absolute Percentage Error (MAPE)	4.10%	< 7.0%	PASSED
Inference Execution Latency	1.38 milliseconds	< 5.0 ms	PASSED
Early Stopping Convergence	Iteration 112 of 300	Plateau on test RMSE	Complete

4.4 Security, Privacy & Data Guarantees

- Zero Key Custody: The server never sees or stores user wallet private keys.
- Local Telemetry Isolation: Raw hardware metrics stay on local loopback (127.0.0.1).
- Zero Personal Data Collection: The agent never accesses personal files, keystrokes, or browser history.
- Safe Actuation: The process whitelist ensures critical operating system services are never disrupted.

Chapter 5: Conclusion & Future Scope

5.1 Project Summary & Key Engineering Achievements

GreenLedger bridges the gap between hardware monitoring and decentralized carbon incentives. By combining a lightweight Windows telemetry agent, a fast sub-2ms XGBoost power inference model, safe automated system tuning, and an ERC-1155 smart contract on Ethereum Sepolia, the system turns abstract energy savings into permanent, verified digital achievements.

5.2 Technical & Academic Disclosures

Academic Disclosure: Power numbers in GreenLedger are machine learning estimates calculated from live hardware performance counters, not physical current shunt measurements. All Ethereum Sepolia tokens and badges exist on an educational testnet and have zero monetary value.

5.3 Future Development Roadmap

- Signed Kernel Diagnostics: Adding an optional signed kernel driver for physical Intel RAPL current telemetry on supported motherboards.
- Layer-2 EVM Rollup: Deploying on Arbitrum or Base for low-cost mainnet badge minting.
- University & Enterprise Fleet Dashboard: Aggregated carbon reporting for university computer labs and office fleets.

References & Technical Bibliography

- [1] Kansal, A., Zhao, F., Liu, J., Kothari, N., & Bhattacharya, A. A. (2010). Virtual machine power metering and provisioning. In Proceedings of the 1st ACM Symposium on Cloud Computing (SoCC '10), pp. 39–50.
- [2] Fieni, G., Rouvoy, R., & Seinturier, L. (2024). PowerAPI: A Python framework for software-defined power meters. Journal of Open Source Software (JOSS), 9(95), 5892.
- [3] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16), pp. 785–794.
- [4] Jirakanchana, T., et al. (2021). Real-time blockchain framework for automated measurement, reporting, and verification in carbon trading. IEEE Access, 9, pp. 102450–102462.
- [5] Sorensen, M. (2023). Tokenized environmental assets and structured pools using multi-token standards. In IEEE International Conference on Blockchain and Cryptocurrency (ICBC '23), pp. 1–8.
- [6] United States Environmental Protection Agency (EPA). (2022). eGRID2022 Technical Summary: GHG Emission Factors for Electricity Generation. Washington, DC.
- [7] Radomski, W., Cooke, C., Castonguay, P., Therien, J., & Binet, E. (2018). EIP-1155: Multi Token Standard. Ethereum Improvement Proposals, no. 1155.
- [8] Ethereum Foundation. (2018). EIP-1193: Ethereum Provider JavaScript API. Ethereum Improvement Proposals, no. 1193.
- [9] OpenZeppelin. (2023). ERC-1155 Multi-Token Contract Standard Implementation (v5.0). <https://docs.openzeppelin.com/contracts/5.x/erc1155>